Aegis architecture and deployment handover

Prepared for Swamy. This is the single maintained Aegis handover: current architecture, OCI resources, model placement, running versions, document workflows and operator release instructions. The live environment was inspected read-only on 15 October 2026 at 22:26 IST; the following facts describe that snapshot.

Aegis is running on Oracle Cloud Infrastructure in Mumbai. The web application, API, agents, worker and CPU reranker run on OKE. Oracle Autonomous AI Database 26ai stores application records, the durable job queue and native vector knowledge. OCI Object Storage holds the documents and saved result files.

The three active datasets currently use Google Gemini for embeddings, chat, extraction and OCR. These model calls leave the OKE backend for the configured Google API. The older laptop Ollama option remains registered, but its containers are stopped and the relay endpoint did not respond during this inspection.

Runs where	Components currently used
OCI OKE	Next.js web app, Caddy HTTPS, FastAPI, LangGraph agents, durable worker and CPU reranker
OCI Autonomous AI Database	Application records, access, durable job queue and Oracle native vector knowledge
OCI Object Storage	Original documents, prompts, parsed content and saved JSON / Parquet results
Google cloud API	Gemini embeddings, chat, extraction and dataset-aware OCR for all three active datasets
Local laptop	Retained Ollama / Qwen / Nomic option; containers stopped and relay endpoint unreachable at the check

The public HTTPS application returned 200. Both application deployments had one ready replica, and all five application containers were ready with zero restarts. The inspection did not deploy, restart, reconfigure, upload documents or invoke model inference. The manager branch is ags_sprint1. The immutable code freeze is aegis-freeze-2026-10-15. Deployment instructions in later sections are for a future authorized release; none were executed while preparing this document.

Recently finalized Mural architecture

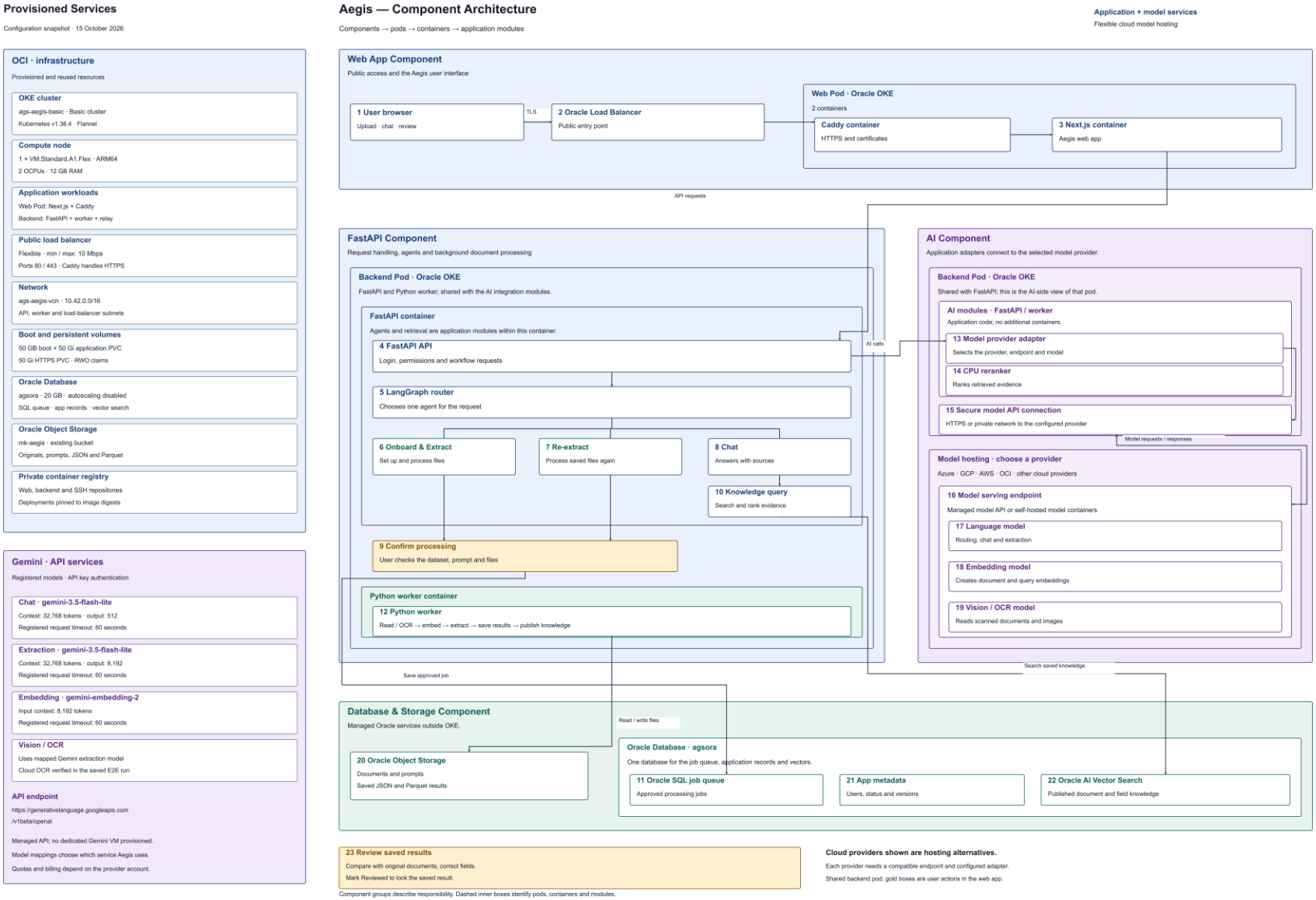


Figure 1 The existing Mural component diagram finalized on 15 October 2026, reused at full resolution with its corrected text spacing. Gemini is the active model path; other providers in the diagram are alternatives. Zoom in for component labels.

Verified OCI resources

These values were read from OCI and Kubernetes during the inspection. The database is the managed Autonomous AI Database service, not the stopped Oracle Free container on the laptop. Its live database version is 26ai and its compute allocation is 1 ECPU.

The two persistent claims each report 50 Gi in Kubernetes and are backed by OCI block volumes reporting 50 GB each. The worker also has a 50 GB boot volume, for 150 GB of allocated boot and block volumes. Database storage is a separate 20 GB allocation.

OCI resource	Live configuration	Observed state
Region and tenancy	Mumbai ap-mumbai-1; maniatramco	Authenticated access
OKE cluster	ags-aegis-basic; Basic; Kubernetes v1.36.4	ACTIVE
Node pool	ags-aegis-a1-node-pool; 1 node	ACTIVE
Compute	VM.Standard.A1.Flex; ARM64; 2 OCPUs; 12 GB RAM	RUNNING / Ready
Autonomous AI Database	ags_ora_db / agsora; Oracle 26ai; 1 ECPU; 20 GB	AVAILABLE
Database configuration	Free Tier flag true; compute/storage autoscaling off; mTLS required	Read connection verified
Object Storage	mk-aegis; Standard; namespace bmyljaecivrj	Read configuration verified
Public load balancer	92.4.67.183; flexible 10 Mbps minimum and maximum	ACTIVE
Application state	application-state; 50 Gi claim; 50 GB OCI volume	Bound / AVAILABLE
HTTPS state	https-state; 50 Gi claim; 50 GB OCI volume	Bound / AVAILABLE
Node boot volume	50 GB; 10 VPUs per GB	AVAILABLE
Private image registry	ags-aegis-web, ags-aegis-backend, ags-aegis-ssh	All private

The bucket currently reports ObjectRead public access and versioning Disabled. Those existing settings were observed and left unchanged. The database Free Tier flag does not establish that the complete OKE environment has zero cost.

Running containers and storage connections

Deployment aegis-web contains web and https. Deployment aegis-api-worker contains api, worker and model-relay. Each deployment has one desired replica and one ready replica. Both use Recreate for updates.

The table shows configured resource requests and limits. A CPU value of 250m means one quarter of a core. These values are not current utilization; the cluster Metrics API was unavailable during the inspection.

Container	CPU and memory request	CPU and memory limit
api	250m CPU; 768Mi	1 CPU; 2Gi
worker	500m CPU; 2Gi	1500m CPU; 4Gi
model-relay	25m CPU; 32Mi	100m CPU; 128Mi
web	150m CPU; 512Mi	1 CPU; 1536Mi
https	50m CPU; 64Mi	250m CPU; 256Mi

1 Database connection. The active destination is OCI database agsora through wallet alias agsora_low, using mTLS. The configured search provider is oracle and the queue provider is database.

2 Content connection. The active content destination is OCI Object Storage bucket mk-aegis in namespace bmyljaecivrj. Originals, parsed content, prompts and versioned results keep their saved storage routes.

3 Persistent application state. API and worker share /data on application-state. It holds protected configuration, routes, locks, archives, model files and bounded application cache. Wallet and OCI credential mounts are separate.

4 HTTPS state. Caddy uses the separate https-state claim for persistent certificate state. Only the public entry is exposed for normal user access; FastAPI is an internal ClusterIP service on port 8000.

One node and one replica per deployment describe the current installation. A future application rollout can interrupt service; no rollout was performed for this documentation update.

Current AI models and local connections

All three active datasets map their Embedding, Chat and Extraction defaults to the Gemini registrations below. Dataset-aware OCR also resolves to the mapped Gemini extraction registration. Global Ollama defaults remain in configuration, but they do not override these saved dataset mappings.

Role	Mapped model	Where it executes
Embedding	gemini-embedding-2	Google Gemini API
Chat	gemini-3.5-flash-lite	Google Gemini API
Extraction	gemini-3.5-flash-lite	Google Gemini API
OCR for mapped datasets	gemini-3.5-flash-lite	Google Gemini API
Candidate reranking	Configured CPU reranker	OCI API / worker; model files on shared state

1 Gemini connection. The saved endpoint host is generativelanguage.googleapis.com. Embedding uses gemini-embedding-2; chat and extraction use gemini-3.5-flash-lite.

2 Saved model limits. The embedding registration has an 8192-token context limit. Chat has 32768 context tokens and 512 output tokens; extraction has 32768 context tokens and 8192 output tokens. These are saved application settings, not a vendor capacity claim.

3 Reranker location. The configured reranker is enabled in local CPU mode inside the OCI backend, with files at /data/models/reranker. Embeddings and generated answers still use the mapped Gemini API.

4 Retained laptop option. Qwen3 chat/extraction and Nomic embeddings remain enabled registrations. The local Ollama container and earlier laptop application/database/index containers are stopped. The internal Ollama relay endpoint was unreachable.

5 OCR behavior. For these Gemini-mapped datasets, OCR uses the saved Gemini extraction model. The retained Qwen vision setting belongs to the local alternative path; it is not the current dataset OCR model.

This check read registrations and mappings without issuing a new Gemini inference request or restarting local models. Model availability and provider quota were not tested by making a paid processing call.

Developed document and answer workflows

1 Workspace and access. Administrators manage users, passwords, roles and dataset assignments. The API checks the signed-in user and selected dataset/document scope before access.

2 Dataset and prompt setup. A dataset maps its model roles. Prompt templates define the fields and validation rules for extraction. Template versions remain associated with saved results.

3 Confirmed document upload. The user confirms the target dataset, documents and required external-processing consent. Original bytes are saved in OCI Object Storage and durable jobs are recorded in Autonomous AI Database.

4 Parsing and indexing. The OCI worker reads the originals, parses text and uses the mapped Gemini model for OCR when needed. Gemini embeddings are published into Oracle native vector knowledge with document and page references.

5 Structured extraction. The worker retrieves and reranks evidence, calls Gemini with the saved prompt, validates the structured response, then saves versioned JSON and Parquet results and publishes result knowledge.

6 Review and export. Users compare saved fields with original documents, correct values and mark a result Reviewed to save and lock it. Exports use saved data. Re-extract creates another result while preserving earlier versions.

7 Unified Chat. Every question follows access and scope checks, hybrid vector and keyword retrieval, CPU reranking, Gemini answer generation, and answer/citation validation. Aegis help follows the same retrieval path. Exact counts use complete authorized catalog facts.

8 Recovery. Accepted worker jobs persist beyond the browser. Progress comes from saved job state. Retries recover the accepted workflow and knowledge publication rather than silently submitting duplicate processing.

The inspection verified configuration and service readiness. It did not run a new upload, extraction, chat question, account change or recovery action.

Versions observed in running services

The application versions below were read from the running containers. Kubernetes and Oracle Database versions were read from live OCI and node information. This page intentionally focuses on deployed technologies and omits package and library dependency inventories.

Running technology	Observed version	Location
Oracle Autonomous AI Database	26ai	OCI managed database
OKE Kubernetes	v1.36.4	OCI Mumbai
Worker operating system	Oracle Linux Server 9.8	OCI ARM64 node
Node.js	22.23.3	OCI web container
Next.js	15.5.27	OCI web container
React	19.3.0	OCI web container
Python	3.12.15	OCI API and worker image
FastAPI	0.142.2	OCI API container
LangGraph	1.2.14	OCI backend agent runtime
Caddy	2.10.2	OCI HTTPS sidecar

The currently mapped model identifiers are gemini-embedding-2 and gemini-3.5-flash-lite. Cloud model identifiers are saved provider selections rather than installed weight versions. Full deployed image references are recorded in the final section of this document.

Network and operating boundaries

The VCN is ags-aegis-vcn with CIDR 10.42.0.0/16. Its API, worker and load-balancer subnets are 10.42.0.0/24, 10.42.1.0/24 and 10.42.2.0/24. The load balancer exposes ports 80 and 443 at 92.4.67.183. The application uses HTTPS and returned the HSTS header.

The internal API service uses port 8000. The retained Ollama service uses port 11434, and the restricted SSH relay service uses NodePort 32022. The relay sidecar is ready, while the laptop model endpoint is unavailable. No model port was opened or network policy changed during the inspection.

The database is available, mTLS is required, and both database compute and storage autoscaling are disabled. The database records the queue and native vectors in the same managed service. The bucket is Standard storage; original documents and complete result files remain in Object Storage.

OCI API calls were GET/list operations. Kubernetes reads inspected nodes, deployments, pods, services and persistent claims; read-only commands inside existing containers checked versions and configuration. The application catalog query explicitly used a read-only database transaction and bypassed cache writes.

The inspected installation has a single worker node and one replica per deployment. The backend release strategy is Recreate, so an application rollout can briefly interrupt the API and worker. Do not promise uninterrupted releases or independent scaling without redesigning shared persistent state.

For a daily pause, first finish chat and all queued or running processing. An application replica pause leaves the worker VM allocated. Stopping the existing Compute VM through OCI Console stops both application pods and preserves the disks; start that same VM and wait for application readiness. Stopped compute does not remove storage, registry, load balancer or database allocations.

Docker Desktop, laptop Ollama and the SSH tunnel are needed only for a deliberately selected laptop model mapping. The three inspected active datasets use Gemini and do not require that tunnel. Do not apply the separate local Compose stack to the existing cloud workspace.

Read-only diagnostics: kubectl get deployments,pods, then inspect events and the relevant container logs using the explicit destination kubeconfig, context and namespace. Keep support output containing private application data outside Git.

Prepare a separate OCI destination

AI can assist with a deployment when it has authorized destination access and the complete installation handoff. Tenant details alone do not install the application. The checked-in code release script updates prepared workloads; the historical ags_oke provisioner and renderer contain pilot-specific assumptions and are not a general empty-tenancy installer.

Use independent destination data, storage, secrets and runtime state. Retain the workspace encryption key across later releases. Do not copy the original tenancy wallet, credentials or user database into another workspace, and do not use START FRESH on the live workspace to prepare a different tenancy.

Destination prerequisite	Required configuration
Access and registry	Destination kubeconfig and context; registry hostname and tenancy namespace; push access and cluster image-pull credentials
OKE application	Namespace aegis-oci; aegis-web with web container; aegis-api-worker with api and worker containers; required supporting containers
Database and files	Initialized destination Oracle database/schema, wallet when required, native vectors and content bucket
Persistent state	Shared API/worker state, runtime configuration and CPU reranker files at /data/models/reranker
Secrets and models	New encryption key and bootstrap token; provider credentials, reachable endpoints and supported model limits
Public access	HTTPS address, certificates and destination networking

1 Initialize the administrator. On the empty initialized database, open Create your workspace and use its private bootstrap token. Create a password of at least 12 characters. Existing sign-in means the destination already has users; inspect it before resetting anything.

2 Register and map models. Register separate Embedding, Chat and Extraction roles, test and save them, then map an active dataset. The current live examples are gemini-embedding-2 and gemini-3.5-flash-lite. New users need explicit dataset access.

3 Verify one real workflow. Use a harmless small document and JSON Schema prompt. Confirm upload and any external-processing notice, wait for indexing, extract, compare with the original, mark Reviewed and ask one cited question. Verify the saved result after refresh.

Prepare the source and release destination

Use a clean checkout of the intended immutable tag and record its commit. The existing freeze is aegis-freeze-2026-10-15. Check Docker, kubectl and destination access before building; use the complete repository as the source directory.

The release script enables Oracle and OCI and disables the optional native cloud-model and local-embedding bundles. Gemini uses the existing HTTP-compatible transport. A destination adapter that requires an optional SDK or sentence-transformers needs those packages built and verified explicitly.

```
# Run from the clean repository root in PowerShell.
$taskSource = (Get-Location).Path
$taskDeploy = Join-Path $taskSource 'ags_deploy/deploy-aegis-code.ps1'
$taskKubeconfig = 'C:\path\destination-kubeconfig.yaml'
$taskContext = 'YOUR_DESTINATION_CONTEXT'
$taskRegistry = 'YOUR_REGISTRY_HOST'
$taskRegistryNamespace = 'YOUR_TENANCY_NAMESPACE'
$taskNamespace = 'aegis-oci'
$taskPlatform = 'linux/arm64'
$taskRelease = 'YOUR_UNIQUE_RELEASE_TAG'

docker version
kubectl version --client
docker login $taskRegistry
kubectl --kubeconfig $taskKubeconfig config get-contexts
kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  -n $taskNamespace get deployments,pods
```

Replace every placeholder, confirm the intended cluster/context and namespace, and resolve missing or unhealthy workloads before publishing. linux/arm64 matches the current A1 node; choose linux/amd64 for an x86 destination. Registry login uses that destination account and OCI auth token. Laptop login does not configure cluster pull credentials. Keep tokens, passwords and keys outside commands, source and images.

Publish and verify an application release

Finish active uploads, indexing, extraction, re-extraction and chat first. Preserve coordinated recovery information and the previous image references. Run the following only for an authorized release into the destination verified on the previous page.

```
& $taskDeploy `
  -SourcePath $taskSource `
  -Kubeconfig $taskKubeconfig `
  -Context $taskContext `
  -Registry $taskRegistry `
  -RegistryNamespace $taskRegistryNamespace `
  -Namespace $taskNamespace `
  -Platform $taskPlatform `
  -Release $taskRelease
```

1 Build and record. The script checks the expected workloads/container names, builds the two application images, verifies their architecture and pushes them. It records previous image references and new digests in private ags_deploy/deployment-records receipts. Record the Git tag and commit beside the receipt.

2 Roll out. The script updates api and worker to the same backend image digest, waits for readiness, then updates web and waits. Supporting containers and externally supplied runtime configuration stay in place. Each rollout wait allows up to 15 minutes.

3 Check the result. Confirm both deployments are ready, sign in through HTTPS and complete one focused harmless workflow. Readiness alone does not establish model quality, extraction correctness or recovery readiness. Repeat only a failed step after fixing its cause.

4 Retain the release record. Keep destination/context, source commit, build flags, previous/new image digests and the actual verification outcome. Use a new release tag for later changes. -SkipBuild is only for the exact verified destination tags already built locally.

This script is not a schema migration, initial application installer, infrastructure provisioner or automatic rollback system. A partial failure can leave one successful rollout in place; inspect both actual versions before restoring anything.

Rollback and coordinated recovery

Read the attempted release receipt and recover the recorded previous API, worker and web image references. Restore only deployments changed by that release, after confirming compatibility with the current database and stored configuration. A failed frontend rollout does not itself justify resetting the workspace.

Set the three image variables below from that private receipt. They may differ for API and worker. Keep the explicit destination settings from the release preparation page.

```
$taskPreviousApi = 'PREVIOUS_API_IMAGE_FROM_RECEIPT'
$taskPreviousWorker = 'PREVIOUS_WORKER_IMAGE_FROM_RECEIPT'
$taskPreviousWeb = 'PREVIOUS_WEB_IMAGE_FROM_RECEIPT'

kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  -n $taskNamespace set image deployment/aegis-api-worker `
  "api=$taskPreviousApi" "worker=$taskPreviousWorker"
kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  -n $taskNamespace rollout status deployment/aegis-api-worker `
  --timeout=15m

kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  -n $taskNamespace set image deployment/aegis-web `
  "web=$taskPreviousWeb"
kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  -n $taskNamespace rollout status deployment/aegis-web `
  --timeout=15m
```

Image rollback does not restore database rows, schema, objects or credentials. Cloud recovery requires coordinated Oracle records/queue/vectors, OCI content objects, shared state, wallet and encryption keys. Verify ownership, checksums, results, citations and job reconciliation in an isolated recovery target before reopening access. The local PostgreSQL backup scripts are not an Oracle cloud restore system; no cloud restore drill was performed for this handover.

Maintain the application and approved design

The application contracts are recorded in root AGENTS.md and the release skill at .agents/skills/aegis-release/SKILL.md. Every Unified Chat question must enforce access and scope, retrieve using semantic and keyword evidence, rerank, generate and validate the answer and citations. Exact counts require complete authorized facts. Do not change the processing agents to implement a chat-only feature.

Use ags_frontend/app/workflow-stepper.tsx and its shared stylesheet for real progress. Completed stages are blue with white checks; an active stage uses a navy rotating ring and particles only on its incoming completed connector. Pending stages remain static. Labels and times describe actual server events; never fabricate percentages, stages or completion. Preserve failure states, keyboard inspection, responsive fitting and reduced-motion behavior.

Use ags_frontend/app/agent-logo.tsx for Agent identity and preserve the existing platform mark in the header. Earlier design reference images remain in Git history. The recently finalized Mural diagram is embedded in this document and requires no external assets.

Runtime values come from ags_backend/ags_app/ags_config/runtime.toml; saved model registrations can impose smaller limits. Keep credentials, bootstrap tokens, wallets and deployment receipts out of Git. Source-declared versions and observed container versions are separate facts.

Source area	Responsibility
ags_backend/ags_app	FastAPI, access, agents, worker, provider and storage adapters
ags_frontend/app	Next.js interface, API proxy, shared progress and identity
ags_database	Oracle SQL definitions and migration boundaries
ags_deploy	OCI release, authentication and runtime tooling
ags_local_use	Local runtime profiles, launchers, Oracle Free setup and optional Kubernetes
ags_testing	Backend/frontend tests, fixtures, QA scripts and test containers

For a separate local installation, use ags_local_use/ags_scripts/setup.sh and ags_local_use/compose.yaml, or the Windows Oracle launcher ags_local_use/ags_scripts/start-oracle.ps1. Use independent data and secrets. Those development alternatives do not describe the live OCI topology. Root README is the brief GitHub entry point; this Word file is the single maintained handover.

Source freeze and validation evidence

The final handover branch is ags_sprint1, created from the python source freeze. Tag aegis-freeze-2026-10-15 remains unchanged. The tag identifies source, not proof that the deployed images are byte-for-byte builds of that commit. The source-only manifest at the tag fingerprints 481 inputs; later documentation and folder changes on ags_sprint1 do not redeploy the application.

Recorded check	Actual result
Full backend regression	16 October 2026: 1047 passed, 5 skipped in 1038.96s; temporary SQLite and mock providers
Test folder reorganization	Tests, fixtures and QA tools are under ags_testing; imports, CI, Compose and container paths verified
Frontend unit suite	178 passed; types and Linux standalone build passed. All 27 browser tests listed only
OCI runtime preview	OCI profile and 49 managed settings validated; applied: false
Deployment tooling	Release script syntax and maintainer skill validated; no application rollout run
Live inspection	HTTPS 200, both replicas ready, five containers ready with zero restarts; no inference, uploads or writes
Document consolidation	The recent Mural image is reused; Word pages are rendered and visually checked; links and source boundaries are verified

Local validation used Python 3.12.8 and Node.js 24.19.0; the running OCI versions are listed earlier. Opt-in integration checks, fresh Compose/browser tests, real-model benchmarks, a new cloud deployment and a restore drill are not claimed by this source freeze.

GitHub Actions triggers on main pushes and pull requests. A direct push to ags_sprint1 is not a fresh CI run and does not deploy the application. The 16 October test relocation was validated locally with temporary SQLite and mock providers; no cloud workload, database or storage settings were changed.

Earlier guides, design QA and dated test reports remain available in Git history at documentation commit a33d5fb. They are not additional current handovers. Preserve the inspection timestamp until another authenticated read-only verification supports newer runtime claims.

Deployment image and diagram provenance

The following image references were recorded from the live deployments during the 15 October 2026 inspection. They identify that observation and were not pushed or changed for this documentation work.

```
Web container
bom.ocir.io/bmyljaecivrj/ags-aegis-web
sha256:12336119afcca52b7eb9bbc34329cc967279787398406a637b52d2b4740b31fc

API and worker containers
bom.ocir.io/bmyljaecivrj/ags-aegis-backend
sha256:fe2ea263dd76ef2a1b50a56b2b26cab0eb81da43cbf0c852dd270f5b8d046ae5
Running platform image ID
sha256:25fc43cab4ba8be40549383b803753277d6043a003a84193803182c35096a780

Model relay sidecar
bom.ocir.io/bmyljaecivrj/ags-aegis-ssh
sha256:37189e82533c3e893f261b9735880e74f521f2ee6ce15f4490e3087ea1944870

HTTPS sidecar
docker.io/library/caddy
sha256:4c6e91c6ed0e2fa03efd5b44747b625fec79bc9cd06ac5235a779726618e530d
```

The backend desired image is an index digest; the running image ID is its platform digest. Both are recorded separately. The diagram is the corrected full-resolution image finalized on 15 October and placed on the existing Mural board; its model-hosting section shows supported alternatives while the live mapping is Gemini.

Mural board: https://app.mural.co/t/clouddataai5577/m/clouddataai5577/1789398606376/9cd26abe7fa9fe9b3ae2ba45aaa4a4722ad240f0?wid=0-1792066520255

Original source manifest: https://github.com/Maniatramco/Aegis/blob/aegis-freeze-2026-10-15/ags_docs/RELEASE-MANIFEST.json